

Penetration Testing Report

Written By: Tova Kleinkaufemn

Target Assessment: OWASP Juice Shop Web Application

Target URL: <https://juice-shop.herokuapp.com/#/>

Testing Timeframe: June 2, 2026 – June 7, 2026

Document Version: Final

Final Assessment Type: Black Box

Origin Source IP: 127.0.0.1

Scope Limits: Testing was strictly confined to the URL specified above. No infrastructure or Denial of Service (DoS) testing was performed unless explicitly authorized.

CONFIDENTIALITY NOTICE

This document contains highly sensitive information regarding the security posture and vulnerabilities of the evaluated target infrastructure. It is intended solely for the use of the authorized recipient(s). Unauthorized distribution, copying, or disclosure of this report, in whole or in part, is strictly prohibited.

This assessment was performed based on information and access available at the time of testing. Security threats are constantly evolving, and the absence of a specific finding does not guarantee that the system is immune to all possible cyber threats. The testing team assumes no liability for any security breaches, operational disruptions, or subsequent modifications made to the environment after the delivery of this document.

Summary of Findings

During the assessment, a total of **8 security findings** were identified that pose a material risk to the target environment.

The table below provides a high-level overview of the identified vulnerabilities, categorized by their potential impact. These findings are covered in complete technical detail, including exploitation vectors and remediation strategies, in the *Technical Findings Details* section of this report.

Vulnerability Name	Severity	Location
Finding #1: Information Disclosure	Critical	Language Sidebar
Finding #2: Reflected XSS	Medium	Search Function
Finding #3: SQL Injection	Critical	Login Page
Finding #4: Information Disclosure and Brute-Force Attack	Critical	Authentication Mechanism
Finding #5: Client-Side Parameter Tampering	Medium	Coupon Code / Checkout System
Finding #6: Insecure Direct Object Reference (IDOR)	High	Payment Method Section
Finding #7: File Extension Validation Bypass via Null Byte	High	/ftp Directory
Finding #8: Unrestricted File Upload Leading to Remote Code Execution	High	/complain page

Finding #1 Information Disclosure (In language SideBar)

Severity: Critical

POC:

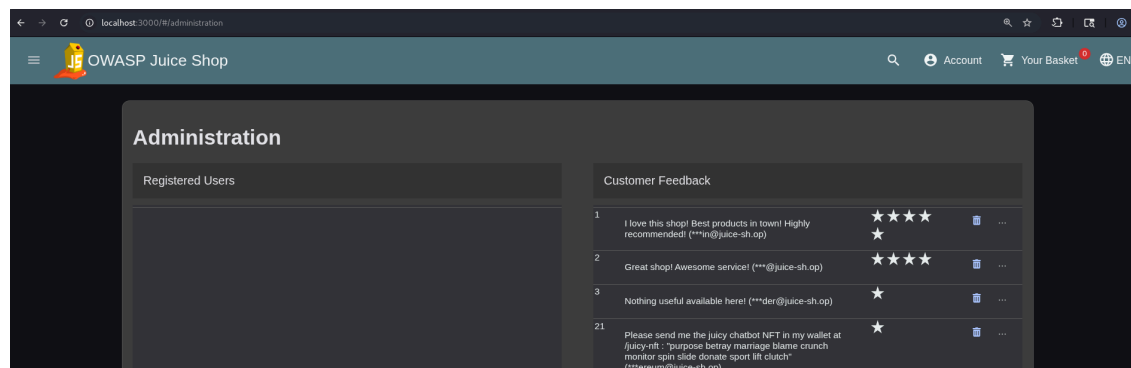
```
localhost:3000/assets/i18n/az_AZ.json

pretty-print

"SECTION_CUSTOMER_FEEDBACK": "Müştəri əks əlaqəsi",
"SECTION_SETTINGS": "Settings",
"LABEL_SHOW_TOOL_CALLS": "Show tool calls in AI chat",
"SECTION_AI_CHAT": "AI Chat",
"TAG_IMPROVED": "Improved",
"ASK_ME_ANYTHING_PLACEHOLDER": "Ask me anything",
"SECTION_SOCIAL_MEDIA": "Bizi Sosial Mediada izləyin",
"LINK_TERMS_OF_USE": "Belə bezdirici şeylərlə maraqlanırs",
"TITLE ADMINISTRATION": "Rəhbərlik",
"SECTION_USER": "Qeydiyyatdan keçən istifadəçilər",
"LABEL_USER": "İstifadəçi",
```

Description: By changing the website's language and intercepting the HTTP request, the localization file was identified as http://localhost:3000/assets/i18n/az_AZ.json.

Reviewing the localization file revealed a reference to the administration panel via the key **"TITLE_ADMINISTRATION": "Rəhbərlik"**. Navigating directly to the inferred path (<http://localhost:3000/#/administration>) bypassed authentication controls entirely, granting unauthenticated access to the administrative dashboard .



Impact:

- **Data Exposure (Confidentiality):** The attacker can view sensitive information.
- **Privilege Escalation:** By accessing the admin interface, a regular user or an anonymous attacker can manipulate user accounts.

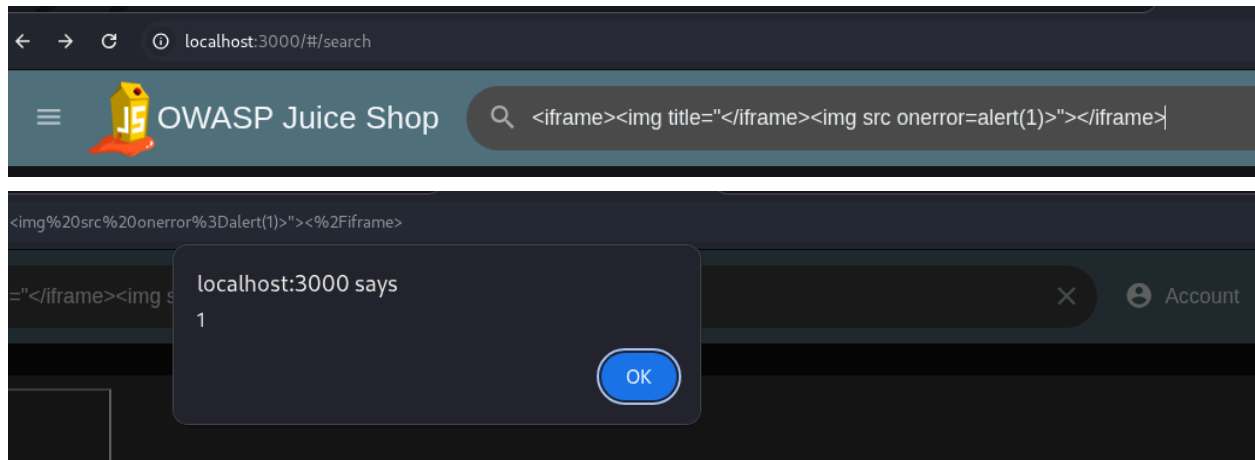
Mitigations:

- **Server-Side Authorization Enforced:** Implement strict role-based access control (RBAC) on the backend APIs.
- **Token Validation:** Ensure every request to administrative endpoints validates the user's JWT
- **Disable Route Enumeration via Localized Files:** Remove sensitive path names from public client-side localization files (.json).

Finding #2 Reflected XSS (In The Search Function)

Severity: Medium

POC:



Description:

A Reflected Cross-Site Scripting (XSS) vulnerability was identified within the application's search functionality. The application accepts user-supplied input via the search parameter and renders it back in the response page without proper validation or sanitization .

Entering this:

```
<iframe><img title="</iframe><img src onerror=alert(1)>"></iframe>
```

The application failed to sanitize this input, leading to the execution of the injected script (`alert(1)`).

Impact:

- **Session Hijacking:** Steal session tokens, cookies or authentication credentials.
- **Account Takeover:** Perform unauthorized actions on behalf of the authenticated user, such as changing account settings or initiating transactions.
- **Defacement & Phishing:** Modify the visual appearance of the page and Redirect users to external, malicious websites designed to deliver **malware**.

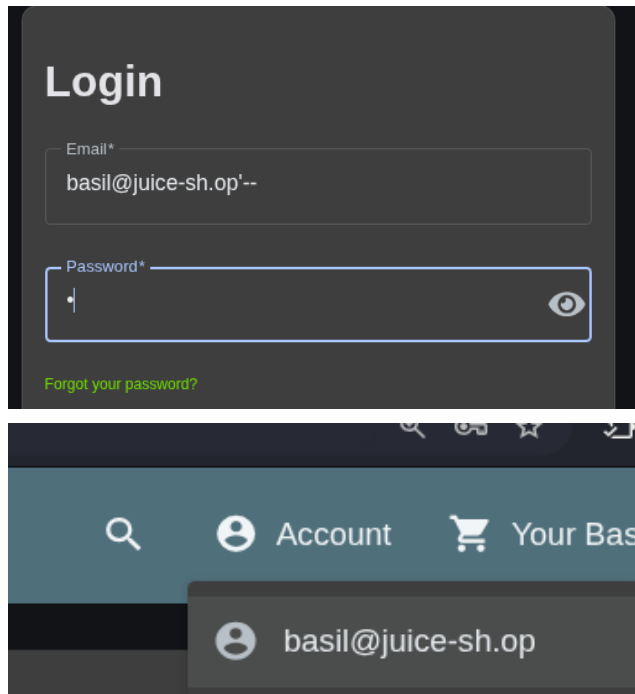
Mitigation:

1. **Context-Aware Output Encoding:** Ensure all user-supplied input is thoroughly encoded before it is rendered on the page.
2. **Input Validation (Wight-list):** Implement strict server-side validation on the search input field..
3. **Content Security Policy (CSP):** Deploy a robust Content Security Policy header to restrict the execution of inline scripts and control the sources from which scripts can be loaded.

Finding #3 SQL Injection (In Login Page)

Severity: Critical

POC:



Description:

An information disclosure vulnerability allowed us to harvest a valid user email (bender@juice-sh.op) from public item reviews. During authentication testing on the login page, submitting a single quote (') in the email field triggered an HTTP 500 Internal Server Error, confirming that input data directly breaks the backend database query structure.

By exploiting this flaw, we successfully bypassed authentication using the following payload in the email field:

bender@juice-sh.op ' --

(The single quote (') escapes the data field context, while the comment sequence (--) instructs the database to ignore the password verification logic entirely)

Impact:

- **Complete Account Takeover:** Access administrative or high-privilege accounts, granting full administrative access to the platform.
- **Data Exfiltration:** Read confidential data stored within the database.
- **Data Manipulation or Destruction:** Modify application data or drop database tables entirely, causing operational disruption.

Mitigation:

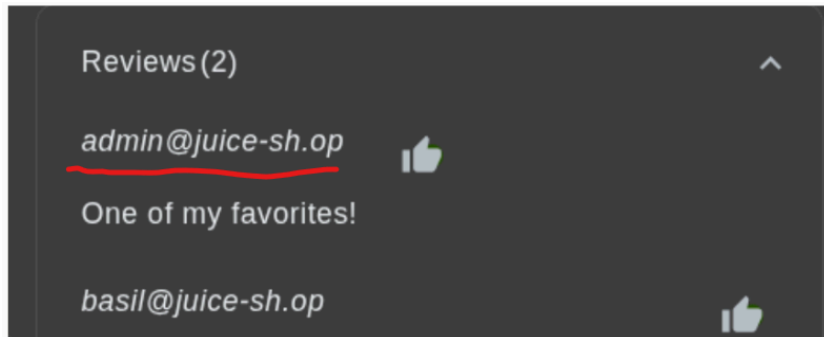
Use Parameterized Queries (Prepared Statements): Ensure that all database interactions use parameterized queries or object-relational mapping (ORM) frameworks.

Finding #4 Information Disclosure and Brute-Force Attack

Severity: Critical

POC:

An Information Disclosure vulnerability was identified within the application's public comments/reviews section, where the administrator's email address is shown.



Description:

Leveraging this acquired credential, a Brute-Force (Credential Stuffing) attack was performed against the login interface using a wordlist of common administrative passwords. Due to the lack of rate-limiting or account lockout controls, the application permitted continuous authentication attempts, allowing us to successfully guess the administrator's password (**admin123**).

View filter: Showing all items		
Request	Payload	Status code ^
5	admin123	200

Impact:

- **Complete System Compromise:** Gain full administrative privileges over the application, allowing total control over user management, application settings, and data.
- **Data Exfiltration and Manipulation:** Access, modify, or delete sensitive user records.
- **Reputational Damage:** Total compromise of the main administrator account.

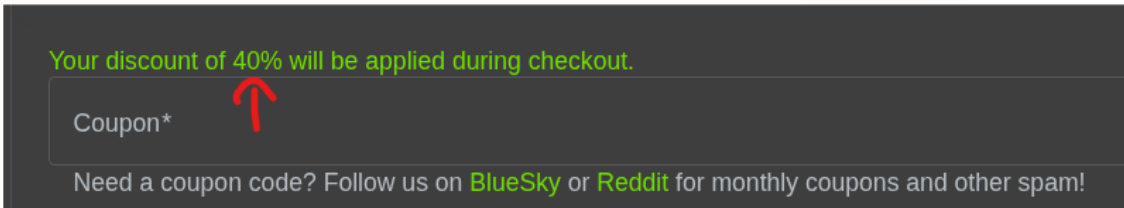
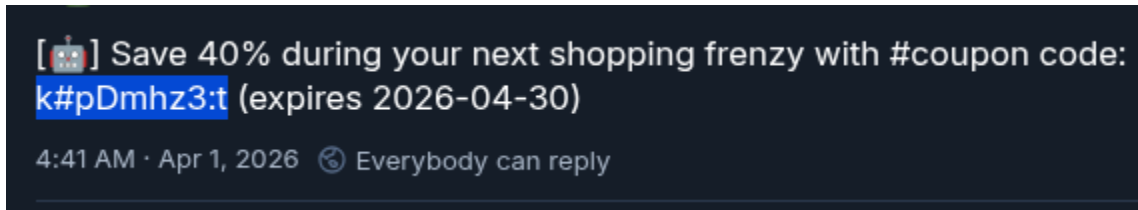
Mitigation:

1. **Implement Rate Limiting and Account Lockout:** Configure mechanisms to temporarily lock user accounts or restrict IP addresses after a specified number of failed login attempts.
2. **Enforce Strong Password Policies:** Require all users, especially high-privilege administrators, to use complex, unique passwords.
3. **Multi-Factor Authentication (MFA):** Mandate Multi-Factor Authentication for all administrative logins.
4. **Sanitize Public Data (Prevent Information Disclosure):** Ensure internal or administrative email addresses are masked or completely removed from public sections.

Finding #5 Client-Side Parameter Tampering (using old coupon code)

Severity: Medium

POC:



Description:

During testing, an expired coupon code dated 30/04/2026 was applied to the shopping cart. The system initially rejected the code, displaying an "invalid coupon" error message. However, by manually modifying the local operating system time on the testing machine back to an active date within the coupon's validity window, the validation logic was completely bypassed.

timedatectl set-time "2026-04-29 23:15:00"

With this we got 40% off the purchase.

Impact:

- **Financial Loss:** Maliciously apply expired, high-value, or promotional seasonal coupons indefinitely to artificially reduce order totals.
- **Bypass Business Limits:** Circumvent time-based restrictions.

Mitigation:

1. **Enforce Server-Side Validation:** Never rely on date or time calculations performed on the client side..
2. **Cryptographic Signatures for Client-Side Data:** If timestamps or active parameters must be transferred from the client, ensure they are wrapped inside a secure, cryptographically signed token.

Finding #6 Insecure Direct Object Reference (IDOR) (Via Payment Method)

Severity: High

POC:

Order Summary

Items	6.86
Delivery	0.99
Promotion	0.00
Total Price	7.85

Place your order and pay

```
9 Connection: keep-alive
0
1 {
    "couponData": "bnVsbA==",
    "orderDetails": {
        "paymentId": "0",
        "addressId": "3",
        "deliveryMethodId": "1"
    }
}
```

Thank you for your purchase!

Your order has been placed and is being processed. You can check for status updates on our [Track Orders](#) page.

Your order will be delivered in 1 days.

Delivery Address
Administrator
0815 Test Street, Test, Test, 4711
Test
Phone Number 1234567890

Description:

During testing, an order was initiated, and a valid saved credit card was selected. The resulting HTTP checkout request was intercepted using a local proxy (Burp Suite). The paymentId parameter, which references the chosen payment method, was changed to a value of 0 before being forwarded to the server.

As a result, the checkout process completed successfully, and the order was placed without any actual monetary transaction taking place.

Impact:

- **Severe Financial Loss:** directly impacting company revenue.
- **Inventory Depletion:** Malicious actors could automate massive checkout requests with tampered parameters.

Mitigation:

1. **Server-Side Transaction Verification.**
2. **Strict Access Control and Parameter Validation.**

Finding #7 File Extension Validation Bypass via Null Byte

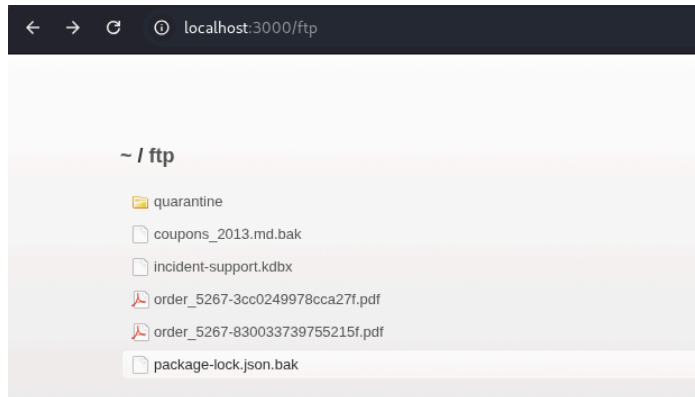
Severity: High

POC:

In robots.txt find a path to /ftp folder

```
User-agent: *  
Disallow: /ftp
```

Navigating to /ftp, we can view the contents of this folder.



Description:

When opening the file package-lock.json.bak, we are unable to read it and receive the following error: **"Only MD and PDF files are allowed"**

OWASP Juice Shop

403 Error: Only .md and .pdf files are allowed!
at verify (/juice-shop/build/routes/fileServer.js:59:18)

We first try adding %00 to bypass the file extension validation, but receive a 403 Unauthorized error.

To fix this, we URL-encode the null byte twice (%2500) and append .md to satisfy the system's requirements: **/ftp/package-lock.json.bak%2500.md**

After this encoding, we successfully download the file and view its content.

(A null byte tells the backend code to terminate the string at that exact position. This allows attackers to bypass file-extension validations.)

Impact:

- **Information Disclosure** - attackers can bypass file extension restrictions to access and download sensitive configuration files .
- **Supply Chain Exposure** -this allows attackers to map out the environment and look for known vulnerabilities (CVEs) .

Mitigation:

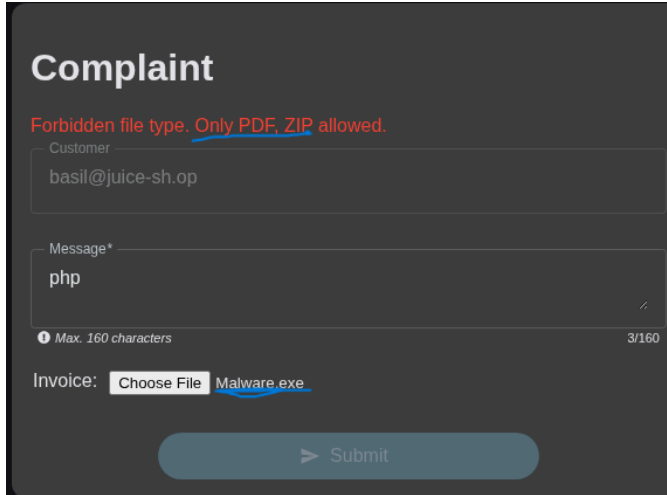
1. Strict Input Validation & Sanitization .
2. Update Runtime Environments.
3. Disable Directory Listing.
4. Use white list for user input.

Finding #8: Unrestricted File Upload Leading to Remote Code Execution (/complain page)

Severity: High

POC:

An initial attempt to upload a file with a **.exe** extension was blocked by the application, returning an error stating that only ZIP and PDF formats are permitted.



The screenshot shows a web form titled "Complaint". At the top, there is an error message in red text: "Forbidden file type. Only PDF, ZIP allowed." Below this, there are three main input sections. The first is labeled "Customer" and contains the email address "basil@juice-sh.op". The second is labeled "Message*" and contains the text "php". Below the message field, there is a character count: "Max. 160 characters" and "3/160". The third section is labeled "Invoice:" and contains a "Choose File" button and a file named "Malware.exe" which is underlined in blue. At the bottom of the form is a large blue button with a right-pointing arrow and the text "Submit".

Description:

To test for client-side restrictions, the local file extension was changed to **.pdf**, allowing the file selection block to accept it.

The HTTP upload request was intercepted using Burp Suite. Before the payload reached the server, the filename extension was modified back to its original **.exe** format.

The server accepted the modified request and returned an HTTP 200 OK response, confirming a successful upload despite the restricted extension.

Impact:

- **Remote Code Execution (RCE):** By bypassing file restrictions, an unauthenticated attacker can upload a functional PHP Web Shell or Reverse Shell to the server.
- **Malware Delivery:** The vulnerability allows malicious files to be stored on the server.
- **Defacement or Denial of Service (DoS).**

Mitigation:

- **Enforce Server-Side Validation.**
- **Implement Content-Based Verification (Magic Numbers):** Validate the file type by **Content-Type** header.
- **Disable Script Execution in Upload Directories.**
- **Rename Uploaded Files:** Rename all uploaded files to randomly generated alphanumeric strings.